

简单介绍算法中的数学

1.位运算

位运算又叫逻辑运算。

常用的位运算：与（&），或（|），非（~），异或（^）

也加上移位运算，左移（<<）,右移（>>）,//左移并对变量赋值（<<=）,右移并对变量赋值（>>=）。

(1) 与

对应于生活中的“且”逻辑，命题A且命题B，要求：命题A，B同时为真，整个命题才为真。

其中真，用“1”表示，假，用“0”表示。

&	0	1
0	0	0
1	0	1

而整数之间的位运算，是先将其写成二进制，此时，每一位数都是0或1，再从低位向高位做位运算。

例如：3&5。

3的二进制	00000011
5的二进制	00000101
结果为	00000001

结果“000000001”的二进制为1。

所以3&5=1；

```
#include<iostream>
using namespace std;
int main()
{
    int a = 3, b = 5;
    cout << (a&b) << endl;//注意，因位运算的运算优先级问题，要打（ ）。
    return 0;
}
//输出： 1
```

(2) 或

同与运算一样，或运算对应于生活中的或逻辑，命题A或命题B，当命题A，B中只要有一个真，整体命题就为真。

	0	1
0	0	1
1	1	1

3的二进制	00000011
5的二进制	00000101

3的二进制	00000011
结果为	00000111

结果“00000001”的二进制为7.

```
#include<iostream>
using namespace std;
int main()
{
    int a = 3, b = 5;
    cout << (a|b) << endl;
    return 0;
}
//输出: 7
```

&与|在运算上，&是越&越小，|是越|越大，这点与贪心多一起出现在比赛中。

(3) 非

非运算是单目运算符，逻辑是：命题A取非，是命题A的真变假，假变真。

~	0	1
	1	0

~a就是将a的二进制中的0变1，1变0，其中还包括控制a正负的01编码。

3的二进制	00000011
~3	11111100

```
#include<iostream>
using namespace std;
int main()
{
    int a = 3, b = 5;
    cout << (~a) << endl;
    return 0;
}
//输出: -4
//~a==-(a+1)
```

(4)异或

异或是命题A，B不同则为真，否则为假。

^	0	1
0	0	1
1	1	0

3的二进制	00000011
5的二进制	00000101
结果为	00000110

结果“00000001”的二进制为6.

```
#include<iostream>
using namespace std;
int main()
{
    int a = 3, b = 5;
    cout << (a^b) << endl;
    return 0;
}
//输出: 6
```

异或的特殊性质：

1.代数结构。

异或满足交换律与结合律。

即 $x^y=y^x$, $x^{(y^z)}=(x^y)^z$.

其中结合律是快速幂的可行条件（虽然没有必要）

异或具有单位元与逆元

单位元是0，0异或任何数都为任何数本身。

x的逆元，是其本身，即 $x^x=0$ (单位元)

异或无零元。

```
/*

补充：运算的单位元，逆元，零元

设运算为“*”//不一定是乘法
*的对应的运算集合为G，G中的任何元素在*的运算下都为
G中的一个元素，不会超出G，则称G对于*是封闭的。
如：实数R上，+，-，*，都是封闭的，而/不行，因为除0的结果不属于R。

*/
```

*	单位元(e)	逆元	零元 (0)
定义	对于任何元素x,都有 $x^*e=x$.	对于任意元素x,若存在一个y,有 $x*y=$ 单位元e,则y是x的逆元	对于任意元素x, $x*0=0$ 恒成立，0即为该运算的零元，记为0.
例子	R上加法的单位元是0，0加任何数都为其任何数本身。	R上加法，x的逆元是-x, $x+(-x)=0$ (加法的单位元)	集合{R, -inf(负无穷)}上的加法运算，-inf就是加法的零元，任何 $x+(-inf)=-inf$ 。

在逆元与结合律的作用下，异或就有了前缀和。

2.线性。

异或等价于模2加法，就有线性。

该性质引出了异或线性基的算法。

(5) 移位

移位运算符不是逻辑运算符，将a的二进制移位。

	3	5
二进制	000011	000101
>>	000001(1)	000010(2)

	3	5
<<	000110(6)	001010(10)

$a >> 1$ 效果等价于 $a/2$ 向下取整。不过位运算的速度远大于直接除2

$a << 1$ 效果等价于 $a*2$, 速度远大于直接乘2、

以上中, $>> 1$, 表示右移1位, 1改成2表示右移2位。等等

但是上面的a并没有改变值, 若要改值, 可以在移位符后加一个“=”。

```
#include<iostream>
using namespace std;
int main()
{
    int a = 3, b = 5;
    cout << (a << 1) << " " << (b >> 1) << endl;
    cout << a << " " << b << endl;
    cout << (a <<= 1) << " " << (b >>= 1) << endl;
    cout << a << " " << b << endl;
    return 0;
}
/*
输出:
6 2
3 5
6 2
6 2
*/
```

(6) 位运算应用

位掩码 (Bitmask) —— 状态压缩的核心

什么是位掩码?

用一个整数的二进制位表示「多个状态」, 比如:

用 $mask = 5$ (101) 表示第 0 位和第 2 位被选中 (适合子集、状态标记)

优点: 内存占用极小 (1 个 int 能表示 32 个状态), 运算速度快。

即, 有 n 个元素的集合, $\{a(1), a(2), a(3) \dots a(n)\}$

可以用 n 位的二进制数表示该集合的子集。用第 i 位的 1, 表示有 $a(i)$, 0 是无 $a(i)$ 。

位运算的与, 可表示对应集合的交集, 或, 可以表示对应集合的并集, 非, 可以表示对应集合的补集。

该方法多用于动态规划的状态压缩。

2. bitset

一、bitset 基本用法

bitset 是固定大小的位序列, 在创建时必须指定长度 (编译时确定)。以下是最常用的基础操作:

```
#include <iostream>
#include <bitset>
using namespace std;

int main() {
    // 1. 创建bitset (长度必须是编译时常量)
    bitset<8> b1; // 默认全0: 00000000
    bitset<8> b2(13); // 用整数初始化: 13的二进制是00001101
    bitset<8> b3("10101010"); // 用二进制字符串初始化: 10101010
    bitset<8> b4(0b11001100); // C++14及以上支持二进制字面量
```

```

// 2. 访问和修改位（从右往左是第0位到第n-1位）
b1[0] = 1; // 设置第0位为1: 00000001
b1.set(2); // 设置第2位为1: 00000101
b1.reset(0); // 重置第0位为0: 00000100
b1.flip(1); // 翻转第1位: 00000110

// 3. 常用成员函数
cout << "b2的二进制: " << b2 << endl; // 输出: 00001101
cout << "b2的大小: " << b2.size() << endl; // 输出: 8
cout << "b2中1的个数: " << b2.count() << endl; // 输出: 3
cout << "b2的第2位: " << b2.test(2) << endl; // 输出: 1 (true)
cout << "b2是否全0: " << b2.none() << endl; // 输出: 0 (false)
cout << "b2是否有1: " << b2.any() << endl; // 输出: 1 (true)

return 0;
}

```

二、bitset 转数字的方法

bitset 支持转换为多种整数类型，核心是通过 `to_ulong()`、`to_ullong()` 等成员函数，或手动计算转换。

方法 1: 使用内置成员函数（推荐）

适合转换为标准整数类型，注意数值不能超出目标类型的范围：

```

#include <iostream>
#include <bitset>
using namespace std;

int main() {
    bitset<8> b("10101010"); // 二进制: 10101010 (十进制170)

    // 1. 转换为unsigned long (无符号长整数)
    unsigned long num1 = b.to_ulong();
    cout << "转unsigned long: " << num1 << endl; // 输出: 170

    // 2. 转换为unsigned long long (无符号长长整数, 支持更大数值)
    unsigned long long num2 = b.to_ullong();
    cout << "转unsigned long long: " << num2 << endl; // 输出: 170

    // 3. 转换为int (需要强制类型转换, 注意范围)
    int num3 = static_cast<int>(b.to_ulong());
    cout << "转int: " << num3 << endl; // 输出: 170

    // 4. 处理负数 (bitset本身无符号, 需手动解析为补码)
    bitset<8> b_neg("11111111"); // 8位补码: -1
    int num_neg = static_cast<int>(b_neg.to_ulong());
    // 由于8位有符号数范围是-128~127, 需要调整符号
    if (b_neg.test(7)) { // 第7位是符号位 (1表示负数)
        num_neg = num_neg - (1 << 8); // 减去2^8, 得到补码对应的负数
    }
    cout << "补码转负数: " << num_neg << endl; // 输出: -1

    return 0;
}

```

方法 2: 手动计算转换（灵活控制）

适合自定义转换逻辑，或处理超出标准类型范围的情况：

```

#include <iostream>
#include <bitset>

```



```

}
/*
输入:
3 7
输出:
2187

*/

```

代码简单且速度快，时间复杂度为 $O(\log n)$

快速幂及其相关衍生是常考的知识点，本身难度不大，但难的是看出这题要用什么快速幂。

下面给出快速幂的衍生,同时为了防止数据溢出，对 $1e9+7$ 取模。//目前选修

因快速幂本质是利用运算的结合律，所以可以适配一些有结合律的运算。

(1) 矩阵快速幂

问题：快速求解 $n \times n$ 的矩阵A，求 A^b

```

#include<iostream>
#include<bitset>
#include<cmath>
#include<vector>
using namespace std;
using ll = long long;
const ll mod = 1e9 + 7;
vector<vector<ll>> mul(const vector<vector<ll>>& a, const vector<vector<ll>>& b)//矩阵相乘
{
    ll n = a.size() - 1;
    vector<vector<ll>> res(n + 1, vector<ll>(n + 1, 0));
    for (int i = 1; i <= n; ++i)
    {
        for (int j = 1; j <= n; ++j)
        {
            for (int k = 1; k <= n; ++k)
            {
                res[i][j] = (res[i][j] + a[i][k] * b[k][j] % mod + mod) % mod;
            }
        }
    }
    return res;
}
vector<vector<ll>> qpow(vector<vector<ll>> a, ll b)//矩阵快速幂
{
    ll n = a.size() - 1;
    vector<vector<ll>> res(n+1,vector<ll>(n+1,0));
    for (int i = 1; i <= n; ++i) res[i][i] = 1;
    while (b)
    {
        if (b % 2) res = mul(res, a);
        a = mul(a, a);
        b >>= 1;
    }
    return res;
}
int main()
{
    ll n, b; cin >> n>>b;//矩阵阶数n,次数b.
    vector<vector<ll>> a(n + 1, vector<ll>(n + 1, 0));
    for (int i = 1; i <= n; ++i)
    {
        for (int j = 1; j <= n; ++j)
        {

```

```

        cin >> a[i][j];
    }
}
vector<vector<ll>> res = qpow(a, b);
for (int i = 1; i <= n; i++)
{
    for (int j = 1; j <= n; ++j)
    {
        cout << res[i][j] << " ";
    }
    cout << endl;
}
return 0;
}
/*
输入:
2 5
1 1
1 0
输出:
8 5
5 3
*/

```

复杂度为 $O(n^3 \log b)$

应用:

求第 n 项兔子数列(记为 $a(n)$), $n \leq 1e18$; 对 $1e9+7$ 取模。

$$a(n) = a(n-1) + a(n-2), a(1) = a(2) = 1$$

解:

$$\text{因 } a(n) = a(n-1) + a(n-2)$$

$$\text{再加上 } a(n-1) = a(n-1)$$

$$\begin{bmatrix} a(n) \\ a(n-1) \end{bmatrix} = \begin{bmatrix} 1 \times a(n-1) & 1 \times a(n-2) \\ 1 \times a(n-1) & 0 \times a(n-2) \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \times \begin{bmatrix} a(n-1) \\ a(n-2) \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^2 \times \begin{bmatrix} a(n-2) \\ a(n-3) \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \times \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

于是求 $a(n)$ 为其矩阵的 $n-1$ 次的 $a[1][1]$.

复杂度为 $O(2^3 \log b)$

(2) 特殊矩阵快速幂

因矩阵的代数结构也有极强可自定义性, 一个矩阵的组成由加法, 乘法构成, $A(+, *)$

若将 $(+, *)$ 换成其它运算, 满足结合律, 也可以有矩阵快速幂。

如:

$A(\oplus, *)$ 快速幂 //异或矩阵快速幂 (不太重要)

$A(+, \max)$ 快速幂 //max-plus矩阵快速幂 (比较重要)

以上不详细给出, 学有余力的可自学。

(3) 多项式快速幂

结合快速幂与快速傅里叶变换或其衍生变换

同样仅作参考。

(4) 加法快速幂

因加法同样有结合律, 同样可以用快速幂,

该应用少。

如: `a*b mod p, (a,b<=1e12,p<=1e18)`

直接乘, 有溢出的风险, 可用加法快速幂求出。

再如: 在已有加法的高精时, 可用加法快速幂代替乘的高精, 复杂度也是 $O(n^2)$ 级, 但慢一个小常数级。

4.基础数论

(1) 最大公因数与最小公倍数

这个的概念想必都在小学了解过, 这里不再复述。

统一术语。

1. $a \mid b$ 表示 a 整除 b , 如 $2 \mid 6$, 同样 $a \nmid b$ 表示 a 不整除 b , 如 $2 \nmid 5$, 传递性: $x \mid a, x \mid b$, 有 $x \mid a + b, x \mid a - b$.

2. $\gcd(a, b)$ 表示是 a, b 的最大公因数, 如 $\gcd(6, 9) = 2$, 当 $\gcd(a, b) = 1$ 时, 称 a, b 是互素的。

3. $\text{lcm}(a, b)$ 表示是 a, b 的最小公倍数, 如 $\text{lcm}(3, 4) = 12$

4.

$a \bmod b$ 表示 a 对 b 取模, 说简单就是取余, 如 $5 \bmod 3 = 2, a \equiv b \pmod{c}$ 表示是 a, b 取模相同。如 $4 \equiv 18 \pmod{7}$

在数论中, 1 与 -1 是作为基本的因子 (该事项在代数结构所定下的), 负数在 $\gcd$ 中与取绝对值一样。

如 $\gcd(-2, 6) = 2, \gcd(-2, -6) = 2$

显然, $\text{lcm}(a, b) = a \times b \div \gcd(a, b)$

一下是求 $\gcd(a, b)$ 的方法, 而 $\text{lcm}(a, b) = a \div \gcd(a, b) \times b$ 可用 $\gcd(a, b)$ 快速求出。

1, 显然 $\gcd(0, b) = b$, 因为 0 可以被任何数整除

2,

不妨设 $a \leq b$, 而有 $\gcd(a, b) = \gcd(a, b - a)$, 进而有 $\gcd(a, b) = \gcd(a, b \bmod a)$
而 $b \bmod a$ 严格小于 a , 可以再次套用 $\gcd(b \bmod a, a) = \gcd(b \bmod a, a \bmod (b \bmod a))$
一直这样操作, 直到一方为 0。

证明: $\gcd(a, b) = \gcd(a, b - a), a \leq b$

设 $t = \gcd(a, b), d = \gcd(a, b - a)$
因 $t \mid a, t \mid b$, 有 $t \mid a, t \mid b - a$ (利用整除的传递性)
所以 t 是 $a, b - a$ 的公因子, 而 d 是 $a, b - a$ 的最大公因子, 所以 $t \leq d$
因 $d \mid a, d \mid b - a$, 有 $d \mid a, d \mid b$ (利用整除的传递性)
所以 d 是 a, b 的公因子, 而 t 是 a, b 的最大公因子, 所以 $d \leq t$
综上: $t \leq d, d \leq t$. 有 $t = d$, 即 $\gcd(a, b) = \gcd(a, b - a)$

```
#include<iostream>
#include<vector>
using namespace std;
using ll = long long;
ll gcd(ll a, ll b)
{
    a = abs(a);
    b = abs(b);
    if (a > b) swap(a, b);
    if (a == 0) return b;
    else return gcd(b % a, a);
}
ll lcm(ll a, ll b)
{
    return a / gcd(a, b) * b; //先除后乘, 尽量不溢出
}
int main()
```

```

{
    ll a, b; cin >> a >> b;
    cout << gcd(a, b) << " " << lcm(a,b) << endl;
    return 0;
}
/*
输入:
20 90
输出:
10 180
*/

```

时间复杂度为 $O(\log(\min(a, b)))$

以下代码原理与上述一样但更快3~5倍，因为使用计算机更快的一些运算。

```

ll gcd(ll a, ll b) {
    if (a == 0) return b;
    if (b == 0) return a;
    ll shift = __builtin_ctz(a | b);
    a >>= __builtin_ctz(a);
    b >>= __builtin_ctz(b);

    while (a != b) {
        if (a > b) {
            a -= b;
            a >>= __builtin_ctz(a);
        }
        else {
            b -= a;
            b >>= __builtin_ctz(b);
        }
    }

    return a << shift;
}

```

注:

`__builtin_ctz(x)` 是 GCC/MinGW 专属内置函数，作用是计算整数二进制末尾连续 0 的个数（比如4是100，末尾 2 个 0，返回 2），对负数会出问题，所以记得先取绝对值。其他编译器（如 VS、Clang）不识别该函数，因此需要替换为跨编译器兼容的实现，同时保持 Stein 算法的效率。但比赛时可以直接用

(2) 模运算及其逆

$a \bmod b$ 是 a 对 b 的余，如 $101 \bmod 10 = 1$

不过编程中的 % 不等价与数论中的取模，当 $a \geq 0$ 时相同， $a < 0$ 时，其结果是负的绝对值取模。

所以我们一般用 $(a \% c + c) \% c$ 等价于 $a \bmod c$

我们来看看它的一些运算性质。

$$1, a + b \bmod c = [(a \bmod c) + (b \bmod c)] \bmod c$$

两数之和取模等于分别取模再求和

$$2, a - b \bmod c = [(a \bmod c) - (b \bmod c)] \bmod c$$

两数之差取模等于分别取模再求差

$$3, a \times b \bmod c = [(a \bmod c) \times (b \bmod c)] \bmod c$$

两数之积取模等于分别取模再求积

但是模运算没有直接的除法。

只有以下的：

$a \times d \equiv b \times d \pmod{c}$ ，且 $\gcd(d, c) = 1$ ，有 $a \equiv b \pmod{c}$

以及一个叫逆的。

乘 $\frac{1}{a}$ 在乘法中是乘 a 的逆运算，乘 A 逆矩阵是乘 A 的逆运算。

我们可以做以下定义： $a \times b \equiv 1 \pmod{c}$ ，其中 b 与 a 的互为逆

则有 $\frac{d}{a} \pmod{c} = [(d \pmod{c}) \times (b \pmod{c})] \pmod{c}$

而 $a \pmod{c}$ 有逆的条件是 a, c 互质，即 $\gcd(a, c) = 1$ 。

而 $\frac{d}{a}$ 在 $\pmod{c}$ 有定义要求 $\frac{d}{a}$ 化简的分母 $\frac{a}{\gcd(a, d)}$ 与 c 互质

一般取模的数，大多是质数，该情况较简单，以下讨论以此为主要背景，合数更复杂，可以用扩展欧几里得算法，以后有机会在讲或大家自学。

模质数 p 的优势是它的因子只有 1 与它自己，只要 $p \nmid a$ ，就有 $\gcd(a, p) = 1$ ，即 0 到 $p - 1$ 都有逆元

那如何求模 p 的逆元呢？

(3) 费马定理与欧拉定理

费马定理：对于质数 $p, p \nmid a$ ，则有 $a^{p-1} \equiv 1 \pmod{p}$

则可以看出 $a^{p-2} \times a \equiv 1 \pmod{p}$ ， a^{p-2} 就满足 a 逆元的定义， a^{p-2} 就是 a 的逆元

而 a^{p-2} 可以用快速幂快速求的。

```
#include<iostream>
#include<vector>
using namespace std;
using ll = long long;
const ll mod = 1e9 + 7;
ll qpow(ll a, ll b)
{
    ll res = 1;
    while (b)
    {
        if (b % 2) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
int main()
{
    ll a; cin >> a;
    ll inv = qpow(a, mod - 2);
    cout << inv << " " << inv * a % mod << endl;
    return 0;
}
/*
输入：
3
输出：
333333336 1
*/
```

欧拉函数: $\phi(n)$ 表示0到 $n-1$ 的整数中与 n 互质的数的个数。

例子: $\phi(6) = 2$, 因为0, 1, 2, 3, 4, 5中, 只有1, 5与6互素。

计算: 设 $p_1, p_2, \dots, p_m$ 是 n 的所有质因数, $\phi(n) = n \times (1 - \frac{1}{p_1}) \times (1 - \frac{1}{p_2}) \times \dots \times (1 - \frac{1}{p_m})$

欧拉定理: 对于一个数 a , $\gcd(a, n) = 1$, 则 $a^{\phi(n)} \equiv 1 \pmod{n}$

当 n 是质数 p 使, 变成费马定理。

欧拉扩展定理: 对于一个数 a , 有 $a^b \equiv a^{b \bmod \phi(n) + \phi(n)} \pmod{n}$

$\phi(n)$ 是积性函数: $\gcd(a, b) = 1$ 时, 有 $\phi(a \times b) = \phi(a) \times \phi(b)$.

(4) 求1到 n 的逆元与 $\phi(x)$ //选修

1, 连续数的逆元

有时, 我们要求1到 $p-1$ 的逆, 若一个一个求逆, 要 $O(n \log n)$ 的复杂度。

但我们有 $O(n)$ 的复杂度的方法。

结论: 设 $inv[i]$ 表示 $i \bmod p$ 的逆元, 有 $inv[i] = -k \times inv[r] \pmod{p}$

其中: $k = \lfloor p/i \rfloor, r = p \bmod i$.

其中 $p \bmod i < i$, 从1到 i , 之前就求出了。

证明: 设 $k = \lfloor p/i \rfloor, r = p \bmod i$. 故有 $i \times k + r = p$.

显然: $p \equiv 0 \pmod{p}$

$i \times k + r \equiv 0 \pmod{p}$

等号两边乘 $inv[r] \times inv[i]$

$(inv[r] \times inv[i]) \times (i \times k + r) \equiv inv[r] \times inv[i] \times 0 \pmod{p}$

$inv[r] \times (inv[i] \times i) \times k + (inv[r] \times r) \times inv[i] \equiv 0 \pmod{p}$

$inv[r] \times k + inv[i] \equiv 0 \pmod{p}$

$inv[i] \equiv -inv[r] \times k \pmod{p}$

以下是计算1到 $1e6$ 模 $1e9+7$ 的逆元模板代码:

```
#include<iostream>
#include<vector>
using namespace std;
using ll = long long;
const ll mod = 1e9 + 7;
const ll maxn = 1e6;
vector<ll> inv(maxn + 1, 1);
void init()
{
    for (int i = 2; i <= maxn; ++i)
    {
        inv[i] = (mod - mod / i * inv[mod % i] % mod) % mod;
    }
}
int main()
{
    init();
    for (int i = 1; i <= 10; ++i)
    {
        cout << inv[i] << endl;
    }
    return 0;
}
/*
输出:
1
500000004
333333336
250000002
400000003
*/
```

```
166666668
142857144
125000001
111111112
700000005
*/
```

计算1到n的逆元，在求组合数等场景应用。

2. 连续的欧拉函数

计算连续数的数论函数，一般用欧拉筛，可在 $O(n)$ 的复杂度求像欧拉函数这样的数论函数。

核心逻辑拆解（分步骤理解）：

1. 用三个数组 / 列表辅助：

- `vis[]`：布尔数组，标记某个数是否为合数（false=素数，true=合数）；
- `primes[]`：存储已经找到的所有素数（相当于“素数字典”）；
- `phi[]`：存储每个数的欧拉函数值（顺带计算，不额外耗时）。

2. 遍历每个数 `i`（从2到n）：

- 如果 `i` 是素数（`vis[i]` 为 false），就把它加入 `primes` 列表，且 `phi[i] = i-1`（素数的欧拉函数性质：与自身互质的数是1~i-1，共i-1个）；
- 用 `i` 和 `prime` 里的每个素数 `p` 相乘，筛除合数 `i*p`：
 - 若 `i*p > n`：超出范围，停止筛选；
 - 标记 `vis[i*p] = true`（筛掉合数）；
 - 关键判断：
 - 若 `i % p == 0`：说明 `p` 是 `i` 的最小质因子，因此 `p` 也是 `i*p` 的最小质因子，此时 `phi[i*p] = phi[i] * p`；
 - 若 `i % p != 0`：说明 `p` 是 `i*p` 的最小质因子（且 `p` 与 `i` 互质），此时 `phi[i*p] = phi[i] * (p-1)`；
 - 当 `i % p == 0` 时，必须 `break`！（再往后的素数都比 `p` 大，不是 `i*p` 的最小质因子了，避免重复筛除）。

```
#include<iostream>
#include<vector>
using namespace std;
using ll = long long;
const ll maxn = 1e6;
vector<ll> prime;
vector<ll> phi(maxn + 1, 1);
vector<bool> vis(maxn + 1, 0);
void init()
{
    phi[1] = 1;
    vis[1] = 1;
    for (ll i = 2; i <= maxn; ++i)
    {
        if (!vis[i])
        {
            vis[i] = true;
            prime.push_back(i);
            phi[i] = i - 1;
        }
        for (int j = 0; j < prime.size(); ++j)
        {
            ll p = prime[j];
```

```

        if ( p * i > maxn) break;
        vis[p * i] = true;
        if (i % p == 0)
        {
            phi[i * p] = phi[i] * p;
        }
        else
        {
            phi[i * p] = phi[i] * phi[p];
        }
    }
}

int main()
{
    init();
    for (int i = 1; i <= 10; i++)
    {
        cout << phi[i] << endl;
    }
    return 0;
}
/*
1
1
2
2
4
2
6
4
6
4
*/

```

欧拉筛是数论中重要的算法，有很多算法要用它。

5.期望的计算

期望的计算有4种常用方法：

a.期望的线性

b.期望的递推

c.建立期望方程组,高斯消元解

d.用组合数学直接计算

其中方法 b,c,d是目前选修,这里仅简单讲解a,b.

(1)期望线性

原理:

$$E(x_1 + x_2 + \dots + x_n) = E(x_1) + E(x_2) + \dots + E(x_n)$$

这里不要求 x_i 之间独立.

例题(1):

抛 n 次公平硬币（正面概率 0.5），求正面出现次数的期望。

解:

目标变量 X : 正面出现的总次数。

拆分为 $X = X_1 + X_2 + \dots + X_n$, 其中: $X_i = 1$ （第 i 次抛硬币为正面）， $X_i = 0$ （反面）。

对每个 $X_i = 1, P(X_i) = 0.5, E(X_i) = 0.5$

$$E(X) = E(X_1) + \dots + E(X_n) = n \times 0.5$$

例题(2):

n 种邮票,每一次购买,都会随机等可能的从这 n 种获得一个,每一种邮票都无限供应,不会卖完,求集齐这 n 种邮票的期望购买次数.

解:

设 X 为购买的次数, X_i 表示要准备集齐的第 i 种邮票单独的次数.

$$X = X_1 + X_2 + \dots + X_n.$$

而 X_i 表示要准备集齐的第 i 种邮票单独的次数.说明已经集齐了 $i-1$ 枚邮票, $\frac{i-1}{n}$ 的可能买不到, $p_i = \frac{n-i+1}{n}$ 的可能买到.

$$E(X_i) = \frac{1}{p_i} = \frac{n}{n-i+1}$$

$$E(X) = \sum_{i=1}^n E(X_i) = \sum_{i=1}^n \frac{n}{n-i+1} = n \times \sum_{i=1}^n \frac{1}{i}$$

(2) 期望递推

该方法理解。从题目中明白。

例题(3):

从1到 n 的数随机等可能选一个数 z_1 ,再从1到 z_1 中随机选一个数 z_2 ,重复以上步骤,直到取到1,求要选数的次数的期望。

解:

设 $E_n(X)$ 表示1到 n 的选数次数的期望, $E_n(X)$ 即为所求。

而 $E_{n-1}(X)$ 到 $E_n(X)$ 有以下递推关系:

$$E_n = \frac{1}{n} \times E_n + \frac{n-1}{n} \times E_{n-1} + 1$$

$$\text{可以快速求的: } E_n = \sum_{i=1}^n \frac{n}{n-i+1} = n \times \sum_{i=1}^n \frac{1}{i}$$

例题(4)

地铁上有1到 n 的连续座位,乘客会在其中空的且相邻座位也为空的座位中随机等可能的选一个。设最后坐人的座位数为 X ,求 $E(X)$ 。

解:

设 $E_n(X)$ 为 n 个座位,题目所求的期望。

一开始从1到 n 中随机取,有:

$$E_n = \frac{1}{n} \times \left(\sum_{k=1}^n E_{k-2} + E_{n-k-2} \right) + 1. \text{其中 } i \leq 0, E_i = 0. \text{记为 (1) 式}$$

令 $n=n-1$,有:

$$E_{n-1} = \frac{1}{n-1} \times \left(\sum_{k=1}^{n-1} E_{k-2} + E_{n-k-3} \right) + 1. \text{记为 (2) 式}$$

而

$$\sum_{k=1}^n E_{k-2} + E_{n-k-2} = 2 \times E_{n-2} + \sum_{k=1}^{n-1} E_{k-2} + E_{n-k-3}. \text{记为 (3) 式}$$

利用 (3) 式, 将 (2) 代入 (1) 式, 得

$$E_n = \frac{1}{n} \times (2 \times E_{n-2} + (n-1) \times (E_{n-1} - 1)) + 1$$

递推即可解得。